llama.cpp is the power-user route and, importantly, the inference engine underneath or alongside many tools in the local-model ecosystem.

To make the comparison useful, try to use the same model in all three routes. For this walkthrough, a practical test candidate is the GGUF release of Qwen3-4B from ggml-org on Hugging Face. Its Q4_K_M file is about 2.5GB, uses the Apache 2.0 licence and is small enough to make the experiment accessible on many machines. The repository provides direct instructions for llama.cpp and Ollama, while LM Studio can search Hugging Face repositories and download supported GGUF models.

You can substitute another small instruction model if this one is no longer your best choice when you read this. Keep the model and quantisation constant when comparing runtimes. Otherwise, you are benchmarking two variables at once.

Before you begin: establish a clean baseline

Do four things before installing anything.

First, check that you have at least 10GB of free storage. The test model is much smaller, but runtimes, caches and later experiments need room.

Second, record your hardware: CPU, system RAM, GPU, VRAM or unified memory. You already learned how to do this in Chapter 4.

Third, close heavy applications. Browsers with dozens of tabs, games and video editors can consume exactly the memory you are trying to give your model.

Fourth, prepare one test prompt and use it everywhere. For example:

"Explain why lithium-ion batteries lose capacity over time in 150 words. Use three bullet points at the end for habits that can reduce degradation. If you are uncertain about a claim, say so."

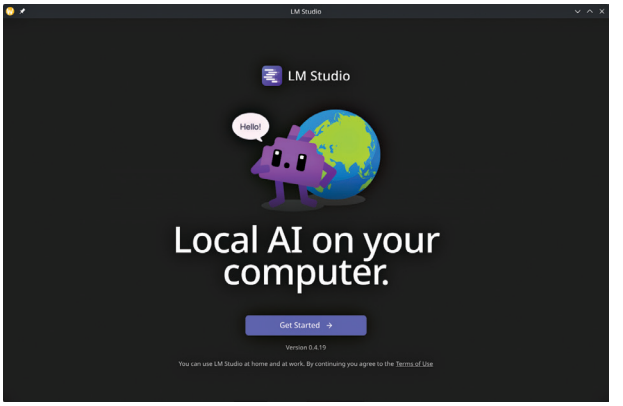
This prompt gives you enough text to judge responsiveness, instruction-following and formatting without turning your first local session into an artificial benchmark marathon.

Route 1: LM Studio, the easiest way

LM Studio is available for Windows, macOS and Linux. It provides model discovery, downloading, a chat interface and model management in one application. Under the hood, it can use llama.cpp runtimes for GGUF models and, on Apple Silicon, also supports Apple's MLX ecosystem. LM Studio currently recommends 16GB or more RAM on Windows and Apple Silicon Macs, although smaller models can work on some 8GB Macs; on Windows, it recommends at least 4GB dedicated VRAM when using a discrete GPU.

Step 1: Install LM Studio

Download LM Studio from its official site and install the version for your operating system. Launch it once while you still have an internet connection. LM Studio separates its app from the model runtimes it uses, so it may need to obtain or update a compatible runtime before a newly downloaded model can run.



Do not download a dozen models during setup. Your first objective is to prove the workflow works.

Step 2: Find the test model

Open the Discover section. LM Studio can search supported Hugging Face models by keyword, a `user/model` repository name or even a full Hugging Face URL.

Search for:

`ggml-org/Qwen3-4B-GGUF`

Open the matching repository and look at the available quantisations. Select Q4_K_M for this experiment. Hugging Face lists that file at roughly 2.5GB. LM Studio's own guidance suggests choosing a 4-bit option or higher when the computer is capable enough.

If the app displays an estimate or device-fit indicator, use it. The objective is comfortable fit, not filling every available byte.

Step 3: Download and verify

Start the download and wait for it to complete. Then go to your local model library or My Models view and confirm that the model appears there.

Know exactly which file you have. "Qwen3" is not enough: record the repository, parameter size and quantisation.

Step 4: Load it into memory

Move to the Chat tab and open the model loader. Select the Qwen3-4B GGUF you downloaded.

For the first run, let LM Studio choose its automatic GPU configuration. Modern versions can determine GPU usage automatically, and the CLI also provides a resource estimator that accounts for context length, GPU settings, flash attention and vision components. If you later use the `lms` command line, `lms load --estimate-only <model>` lets you estimate requirements before loading.

Keep the context modest – around 4K tokens is plenty for our first chat. A huge context window is unnecessary for a 150-word battery question and consumes more memory.